🛰️ 01. FMCW Radar Signal Processing & Target Detection: First-Principles Walkthrough

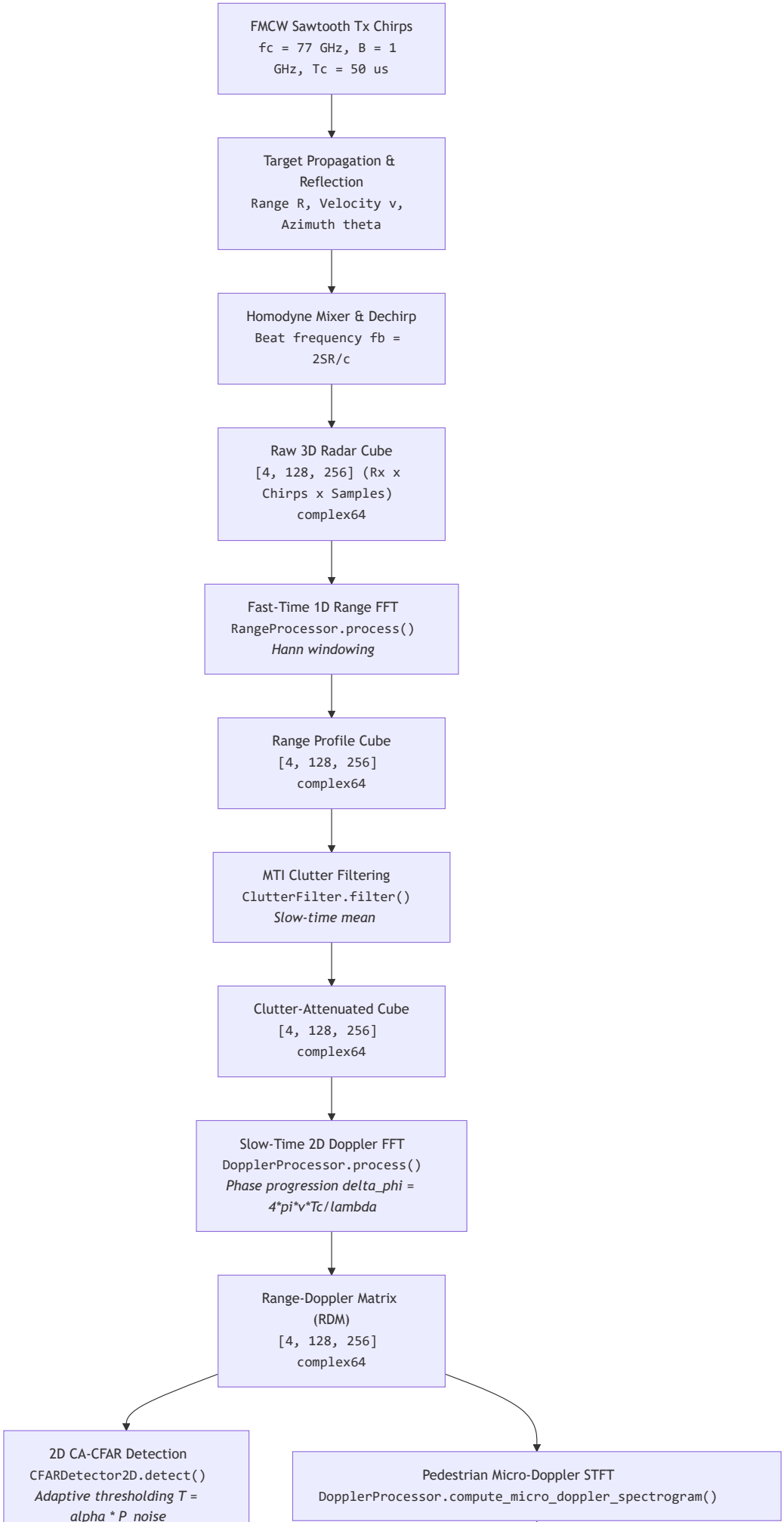
> **BTP Research & Pedagogy Masterclass**
> **Topic:** 77 GHz Automotive mmWave Radar Signal Processing Pipeline
> **Key Disciplines:** FMCW Waveform Theory • Fast-Time Range FFT • MTI Clutter Filtering • Slow-Time Doppler FFT • 2D CA-CFAR • Spatial Array AoA (Capon & MUSIC) • DBSCAN Cartesian BEV Tracking • Micro-Doppler STFT

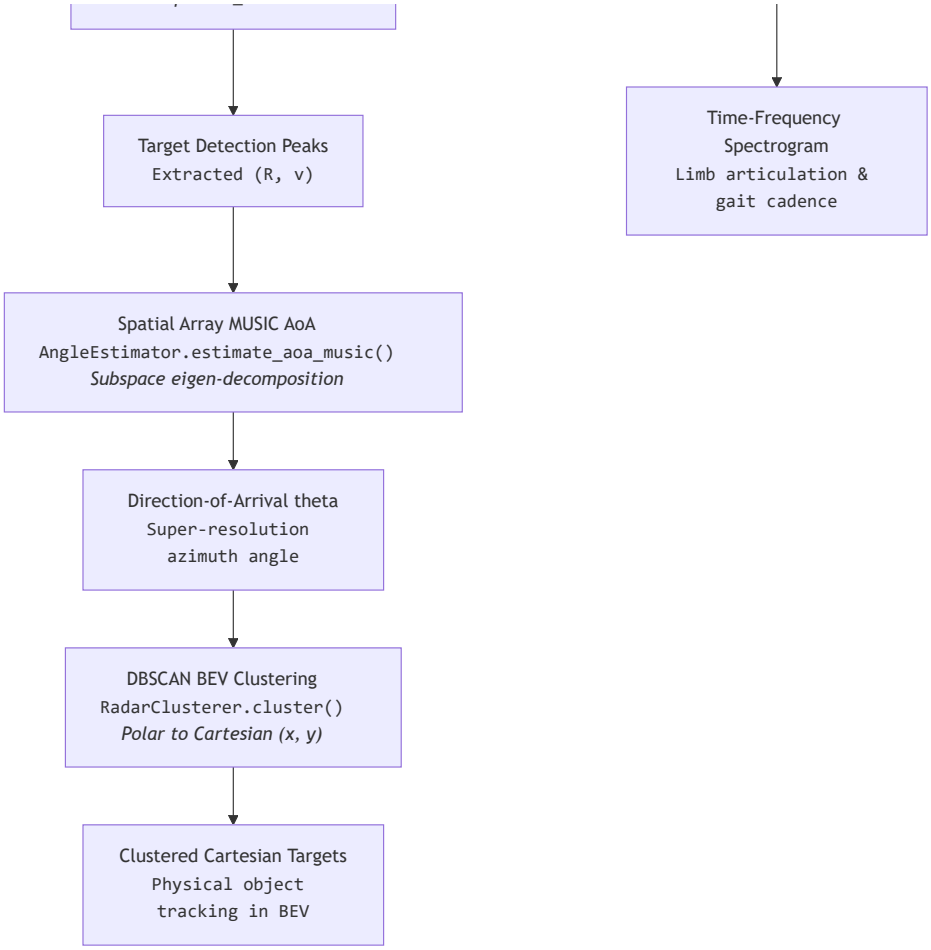
Executive Summary for Academic Review

Frequency-Modulated Continuous-Wave (FMCW) radar is the bedrock of modern automotive active safety, autonomous driving, and robotic spatial perception. Unlike pulsed radar systems, FMCW radars transmit continuous frequency ramps (chirps), enabling simultaneous measurement of **target range**, **relative radial velocity**, and **azimuth angle of arrival (AoA)** with high energy efficiency and millimetric accuracy.

This notebook provides a complete, first-principles derivation and implementation of the classical radar digital signal processing (DSP) chain. Every stage is verified against physical equations and visualized using a publication-grade dark cyber-radar theme:

End-to-End Radar DSP Signal Processing Architecture


FMCW Sawtooth Tx Chirps
fc = 77 GHz, B = 1 GHz, Tc = 50 us
Target Propagation & Reflection
Range R, Velocity v, Azimuth theta
Homodyne Mixer & Dechirp
Beat frequency fb = 2SR/c
Raw 3D Radar Cube
[4, 128, 256] (Rx x Chirps x Samples) complex64
Fast-Time 1D Range FFT
RangeProcessor.process()
Hann windowing
Range Profile Cube
[4, 128, 256] complex64
MTI Clutter Filtering
ClutterFilter.filter()
Slow-time mean
Clutter-Attenuated Cube
[4, 128, 256] complex64
Slow-Time 2D Doppler FFT
DopplerProcessor.process()
Phase progression delta_phi = 4*pi*v*Tc/lambda
Range-Doppler Matrix (RDM)
[4, 128, 256] complex64
2D CA-CFAR Detection
CFARDetector2D.detect()
Adaptive thresholding T = alpha * P_noise
Pedestrian Micro-Doppler STFT
DopplerProcessor.compute_micro_doppler_spectrogram()


Target Detection Peaks
Extracted (R, v)
Spatial Array MUSIC AoA
AngleEstimator.estimate_aoa_music()
Subspace eigen-decomposition
Direction-of-Arrival theta
Super-resolution
azimuth angle
DBSCAN BEV Clustering
RadarClusterer.cluster()
Polar to Cartesian (x, y)
Clustered Cartesian Targets
Physical object
tracking in BEV
Time-Frequency
Spectrogram
Limb articulation &
gait cadence

In [1]:
```
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

# Configure project paths
PROJECT_ROOT = os.path.abspath(".")
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.sim.radar_config import RadarConfig, SPEED_OF_LIGHT
from src.sim.target import RadarTarget, create_standard_traffic_scenario
from src.sim.radar_sim import FMCWRadarSimulator
from src.dsp.range_processing import RangeProcessor
from src.dsp.clutter_removal import ClutterFilter
from src.dsp.doppler_processing import DopplerProcessor
from src.dsp.cfar_detector import CFARDetector2D
from src.dsp.aoa_estimation import AngleEstimator
from src.dsp.clustering import RadarClusterer
from src.dsp.radar_pipeline import ClassicalRadarPipeline

# Configure Dark Cyber-Radar Theme
plt.style.use('dark_background')
plt.rcParams.update({
    'figure.facecolor': '#0a0e17',
    'axes.facecolor': '#111625',
    'axes.edgecolor': '#2a364f',
    'axes.labelcolor': '#00f0ff',
    'xtick.color': '#8b9cb3',
    'ytick.color': '#8b9cb3',
    'grid.color': '#1e283d',
    'grid.alpha': 0.6,
    'text.color': '#e2e8f0',
    'font.family': 'sans-serif',
    'font.size': 10
})

print("✅ Dark Cyber-Radar Theme initialized.")
print("✅ Core Radar DSP libraries loaded from src/.")
```

```
✅ Dark Cyber-Radar Theme initialized.
✅ Core Radar DSP libraries loaded from src/.
```

1. FMCW Radar Physics & Operating Equations



An FMCW radar transmits a linear frequency-modulated (LFM) continuous wave chirp:

$$s_{tx}(t) = A_{tx} \cos\left(2\pi f_c t + \pi S t^2\right), \quad 0 \le t \le T_c$$

where:

- f_c = Center carrier frequency (e.g., $77\ \text{GHz}$, $\lambda = \frac{c}{f_c} \approx 3.9\ \text{mm}$)
- B = Chirp sweep bandwidth (e.g., $1.0\ \text{GHz}$)
- T_c = Active chirp sweep duration (e.g., $50\ \mu\text{s}$)
- $S = \frac{B}{T_c}$ = Chirp frequency slope (Hz/s)

Target Reflection & Dechirping

For a target at range $R(t) = R_0 + vt$, the round-trip propagation time delay is:

$$\tau(t) = \frac{2R(t)}{c} = \frac{2(R_0 + vt)}{c}$$

The received signal is mixed with the transmitted chirp (homodyne dechirping) and low-pass filtered, yielding the intermediate frequency (**IF beat signal**):

$$s_b(t) = A_b \cos(2\pi f_b t + \phi_0)$$

where the **beat frequency** f_b maps directly to physical range:

$$f_b = S\tau = \frac{2SR_0}{c} \implies R = \frac{c \cdot f_b}{2S}$$

Fundamental Resolution & Sampling Invariants

1. **Range Resolution (ΔR)**: Governed purely by chirp bandwidth B:

$$\Delta R = \frac{c}{2B}$$

2. **Maximum Unambiguous Range (R_{max})**: Bound by ADC sampling rate F_s:

$$R_{max} = \frac{F_s c}{2S}$$

3. **Velocity Resolution (Δv)**: Governed by total frame coherent processing interval ($CPI = N_{chirps} T_c$):

$$\Delta v = \frac{\lambda}{2N_{chirps}T_c}$$

4. **Maximum Unambiguous Velocity (v_{max})**: Bound by chirp repetition interval T_c:

$$v_{max} = \frac{\lambda}{4T_c}$$

In [2]:
```
# Instantiate radar hardware specification (Texas Instruments AWR1843
equivalent)
cfg = RadarConfig(
    fc=77e9,              # 77 GHz center frequency
    bandwidth=1.0e9,      # 1 GHz bandwidth -> Delta_R = 15 cm
    chirp_duration=50e-6, # 50 us chirp duration Tc
    num_samples=256,      # Fast-time ADC samples per chirp
    num_chirps=128,       # Slow-time chirps per frame
    num_rx=4,             # 4-element Uniform Linear Array (ULA)
    rx_spacing_factor=0.5,# d = lambda / 2
    sampling_rate=5.12e6  # 5.12 MHz ADC sampling rate Fs
)

# Display radar performance metrics
print("=" * 65)
print("             77 GHz FMCW RADAR SYSTEM PARAMETERS")
print("=" * 65)
print(f"Carrier Frequency (fc)     : {cfg.fc / 1e9:.2f} GHz")
print(f"Wavelength (lambda)        : {cfg.wavelength * 1e3:.3f} mm")
print(f"Sweep Bandwidth (B)        : {cfg.bandwidth / 1e9:.2f} GHz")
print(f"Chirp Duration (Tc)        : {cfg.chirp_duration * 1e6:.1f} us")
print(f"Chirp Frequency Slope (S)  : {cfg.chirp_slope / 1e12:.2f} MHz/us")
print(f"Range Resolution (Delta_R) : {cfg.range_resolution:.4f} m (15 cm)")
print(f"Max Range (R_max)          : {cfg.max_unambiguous_range:.2f} m")
print(f"Velocity Resolution (Dv)   : {cfg.velocity_resolution:.4f} m/s")
print(f"Max Velocity (v_max)       : {cfg.max_unambiguous_velocity:.2f} m/s (+/-
{cfg.max_unambiguous_velocity * 3.6:.1f} km/h)")
print(f"Antenna Array Elements     : {cfg.num_rx} Rx (ULA, d =
{cfg.antenna_spacing * 1e3:.2f} mm)")
print("=" * 65)

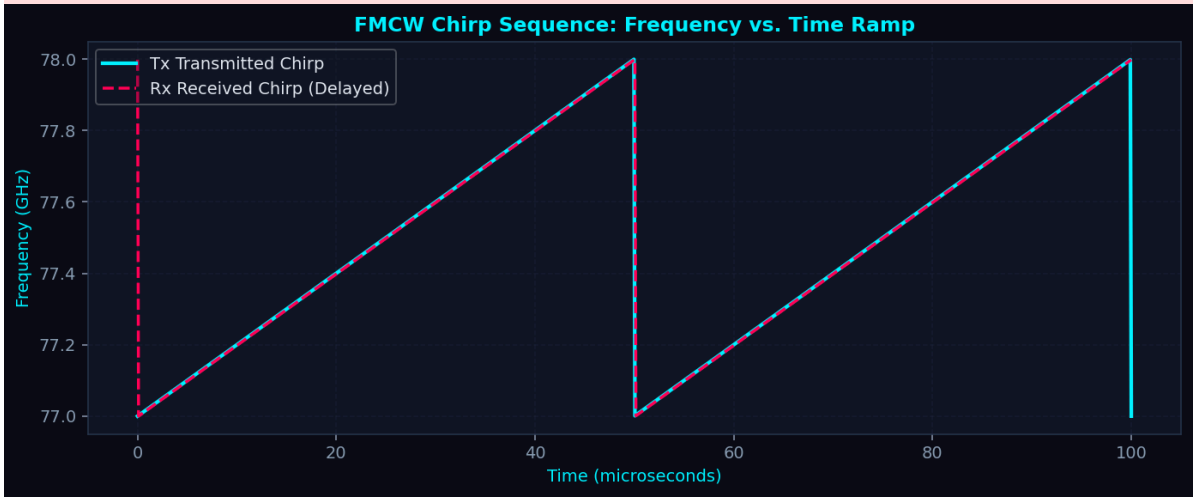
# Visualize FMCW Sawtooth Transmit & Receive Frequency Modulation
t_sweep = np.linspace(0, 2 * cfg.chirp_duration, 1000)
f_tx = np.zeros_like(t_sweep)
tau_demo = 2 * 15.0 / SPEED_OF_LIGHT # 15m delay
f_rx = np.zeros_like(t_sweep)

for i, t in enumerate(t_sweep):
    t_mod = t % cfg.chirp_duration
    f_tx[i] = (cfg.fc + cfg.chirp_slope * t_mod) / 1e9
    t_rx_mod = (t - tau_demo) % cfg.chirp_duration
    f_rx[i] = (cfg.fc + cfg.chirp_slope * t_rx_mod) / 1e9

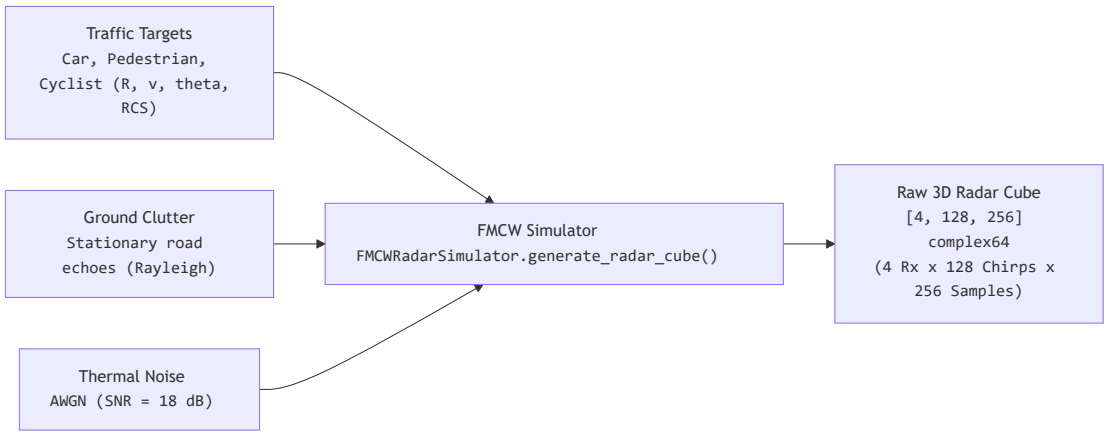
plt.figure(figsize=(10, 4.2), dpi=140)
plt.plot(t_sweep * 1e6, f_tx, label="Tx Transmitted Chirp", color="#00f0ff",
lw=2.2)
plt.plot(t_sweep * 1e6, f_rx, label="Rx Received Chirp (Delayed)",
color="#ff0055", lw=1.8, linestyle="--")
plt.title("FMCW Chirp Sequence: Frequency vs. Time Ramp", fontsize=12,
fontweight="bold", color="#00f0ff")
plt.xlabel("Time (microseconds)", fontsize=10)
plt.ylabel("Frequency (GHz)", fontsize=10)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend(loc="upper left", framealpha=0.3)
plt.tight_layout()
plt.show()
```

```
================================================================
              77 GHz FMCW RADAR SYSTEM PARAMETERS
================================================================
Carrier Frequency (fc)     : 77.00 GHz
Wavelength (lambda)        : 3.893 mm
Sweep Bandwidth (B)        : 1.00 GHz
Chirp Duration (Tc)        : 50.0 us
Chirp Frequency Slope (S)  : 20.00 MHz/us
Range Resolution (Delta_R) : 0.1499 m (15 cm)
Max Range (R_max)          : 38.37 m
Velocity Resolution (Dv)   : 0.3042 m/s
Max Velocity (v_max)       : 19.47 m/s (+/- 70.1 km/h)
Antenna Array Elements     : 4 Rx (ULA, d = 1.95 mm)
================================================================
```

```
<string>:50: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



2. Multi-Target Traffic Scenario & Raw 3D Radar Cube



We construct a realistic automotive traffic scenario containing targets with varying kinematics, radar cross-sections (RCS), and spatial azimuth angles:

Target ID	Object Class	Range (R_0)	Radial Velocity (v)	Azimuth Angle (θ)	RCS (σ)	Kinematic Behavior
#1	**Car (Approaching)**	$12.0\ \text{m}$	$+8.5\ \text{m/s}$ ($+30.6\ \text{km/h}$)	$+18.0^\circ$	$+8.0\ \text{dBsm}$	Approaching in right lane
#2	**Car (Receding)**	$28.5\ \text{m}$	$-14.2\ \text{m/s}$ ($-51.1\ \text{km/h}$)	-12.0°	$+12.0\ \text{dBsm}$	Receding fast in left lane
#3	**Pedestrian**	$8.4\ \text{m}$	$+1.4\ \text{m/s}$ ($+5.0\ \text{km/h}$)	$+28.0^\circ$	$-4.0\ \text{dBsm}$	Walking with micro-Doppler gait
#4	**Cyclist**	$19.2\ \text{m}$	$-4.8\ \text{m/s}$ ($-17.3\ \text{km/h}$)	$+5.0^\circ$	$+1.0\ \text{dBsm}$	Receding along roadway
#5	**Ground Clutter**	Distributed	$0.0\ \text{m/s}$	0.0°	Strong Rayleigh	Stationary road pavement & barriers

Raw 3D Radar Cube Dimensions

The raw ADC output forms a 3D complex matrix:

$$\mathbf{X} \in \mathbb{C}^{N_{rx} \times N_{chirps} \times N_{samples}} = \mathbb{C}^{4 \times 128 \times 256}$$

In [3]:
```
# Define targets and instantiate simulator
targets = create_standard_traffic_scenario()
sim = FMCWRadarSimulator(cfg)

# Synthesize raw complex radar cube with noise and clutter
radar_cube = sim.generate_radar_cube(
    targets=targets,
    snr_db=18.0,
    clutter_ratio_db=10.0,
    include_clutter=True,
    seed=42
)

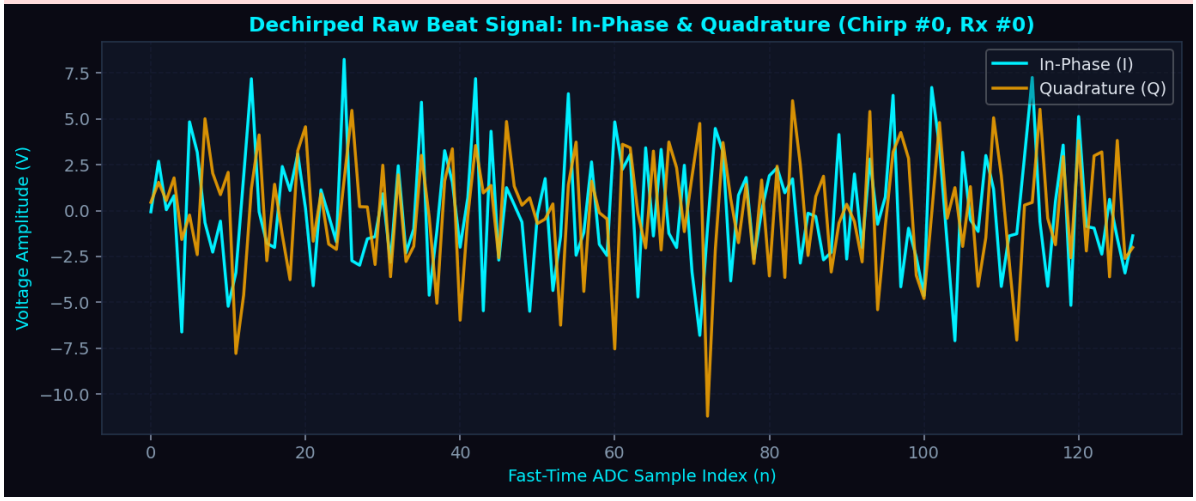
print(f"Synthesized 3D Radar Cube Shape: {radar_cube.shape} | Data Type:
{radar_cube.dtype}")
print(f"Elements: {radar_cube.shape[0]} Rx Antennas x {radar_cube.shape[1]}
Chirps x {radar_cube.shape[2]} ADC Samples")

# Plot Raw Dechirped I/Q Beat Signal for Chirp #0 on Rx #0
chirp_0 = radar_cube[0, 0, :]
sample_idx = np.arange(128) # first 128 fast-time samples

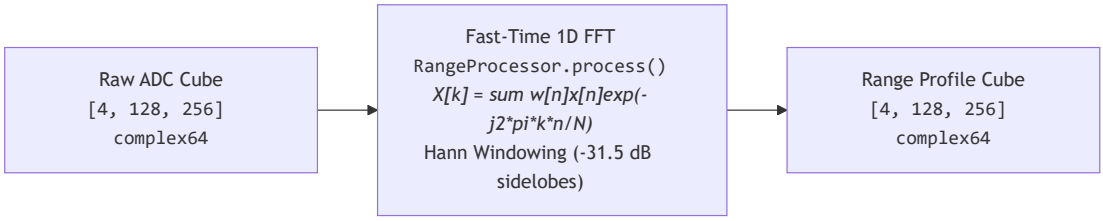
plt.figure(figsize=(10, 4.2), dpi=140)
plt.plot(sample_idx, np.real(chirp_0[:128]), label="In-Phase (I)",
color="#00f0ff", lw=1.8)
plt.plot(sample_idx, np.imag(chirp_0[:128]), label="Quadrature (Q)",
color="#ffaa00", lw=1.8, alpha=0.85)
plt.title("Dechirped Raw Beat Signal: In-Phase & Quadrature (Chirp #0, Rx #0)",
fontsize=12, fontweight="bold", color="#00f0ff")
plt.xlabel("Fast-Time ADC Sample Index (n)", fontsize=10)
plt.ylabel("Voltage Amplitude (V)", fontsize=10)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend(loc="upper right", framealpha=0.3)
plt.tight_layout()
plt.show()
```

```
Synthesized 3D Radar Cube Shape: (4, 128, 256) | Data Type: complex128
Elements: 4 Rx Antennas x 128 Chirps x 256 ADC Samples
```

```
<string>:30: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



3. Fast-Time 1D Range FFT & Sidelobe Suppression



To extract target range, we compute the Discrete Fourier Transform (DFT) along the fast-time sample dimension for each chirp:

$$X_{range}[k] = \sum_{n=0}^{N_{samples}-1} w[n] \cdot x[n] \cdot e^{-j2\pi \frac{kn}{N_{samples}}}$$

Windowing for Spectral Leakage Suppression

A rectangular window has high sidelobes (-13 dB), which can mask weaker targets (such as a pedestrian with -5 dBsm RCS) near a large vehicle ($+10$ dBsm). We apply a **Hann window**:

$$w_{Hann}[n] = 0.5 - 0.5\cos\left(\frac{2\pi n}{N_{samples}-1}\right)$$

which attenuates sidelobes down to -31.5 dB, preventing false alarms and dynamic range saturation.

In [4]:
```
# Compute Range FFT with Hann windowing
range_proc = RangeProcessor(cfg, window="hann")
range_cube, range_axis = range_proc.process(radar_cube)

# Unwindowed (rectangular) version for side-by-side comparison
range_proc_rect = RangeProcessor(cfg, window="rect")
range_cube_rect, _ = range_proc_rect.process(radar_cube)

# Power spectrum across range (averaged over chirps)
pwr_hann = 10 * np.log10(np.mean(np.abs(range_cube[0])**2, axis=0))
pwr_rect = 10 * np.log10(np.mean(np.abs(range_cube_rect[0])**2, axis=0))

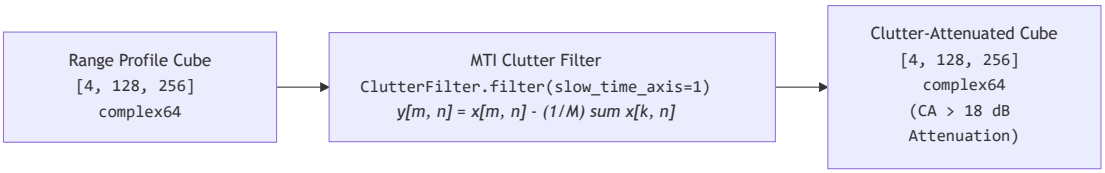
plt.figure(figsize=(10, 4.2), dpi=140)
plt.plot(range_axis, pwr_rect, label="Rectangular Window (High Sidelobes)",
color="#ff5555", lw=1.2, alpha=0.6)
plt.plot(range_axis, pwr_hann, label="Hann Window (Suppressed Sidelobes)",
color="#00ff66", lw=2.0)

# Annotate expected targets
plt.axvline(8.4, color="#ff00ff", linestyle=":", alpha=0.7, label="Target #3
(Pedestrian: 8.4m)")
plt.axvline(12.0, color="#00f0ff", linestyle=":", alpha=0.7, label="Target #1
(Car: 12.0m)")
plt.axvline(19.2, color="#ffaa00", linestyle=":", alpha=0.7, label="Target #4
(Cyclist: 19.2m)")
plt.axvline(28.5, color="#00ff66", linestyle=":", alpha=0.7, label="Target #2
(Car: 28.5m)")

plt.title("1D Range Profile: Windowing & Sidelobe Comparison", fontsize=12,
fontweight="bold", color="#00f0ff")
plt.xlabel("Range (meters)", fontsize=10)
plt.ylabel("Normalized Power (dB)", fontsize=10)
plt.xlim([0, 35])
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend(loc="upper right", framealpha=0.3, fontsize=8)
plt.tight_layout()
plt.show()
```



4. Ground Clutter Suppression via MTI Filtering



Stationary obstacles (road surface, guardrails, bridges) generate massive radar echoes at zero Doppler frequency ($v = 0\ \mathrm{m/s}$). This clutter masks moving dynamic road users.

We deploy **Moving Target Indication (MTI)** via slow-time mean subtraction:

$$y[m,n] = x[m,n] - \frac{1}{M}\sum_{k=0}^{M-1} x[k,n]$$

where m is the chirp index (slow-time) and n is the range bin index (fast-time).

Clutter Attenuation (CA) Metric

The effectiveness of the filter is quantified by the Clutter Attenuation ratio:

$$\mathrm{CA}\ (\mathrm{dB}) = 10\log_{10}\left(\frac{P_{in}}{P_{out}}\right)$$

In [5]:
```
# Apply MTI Clutter Filter
clutter_filter = ClutterFilter(method="mean_subtraction")
filtered_range_cube = clutter_filter.filter(range_cube, slow_time_axis=1)

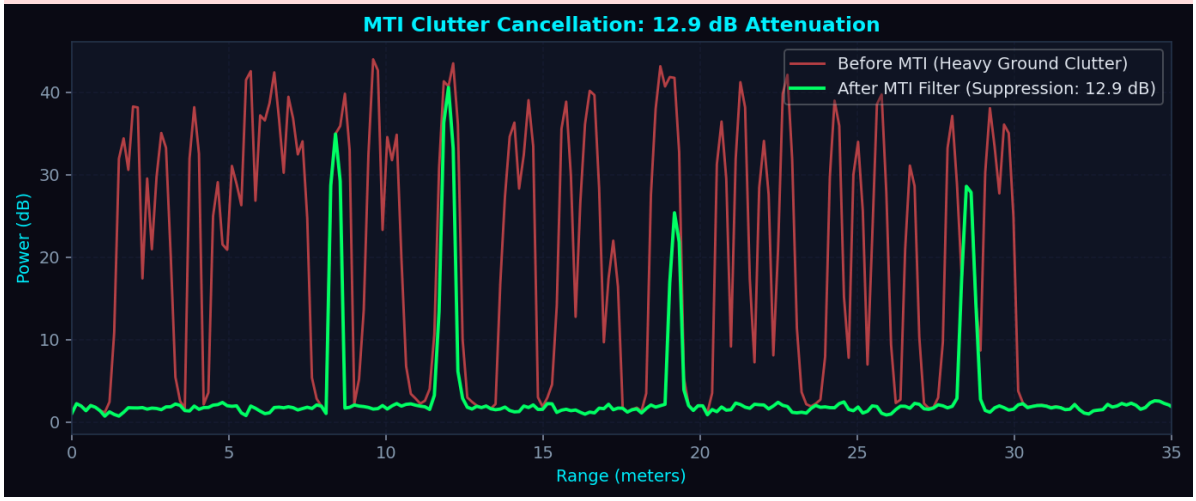
ca_db = ClutterFilter.compute_clutter_attenuation_db(range_cube,
filtered_range_cube)
print(f"[+] Clutter Attenuation (CA) Achieved: {ca_db:.2f} dB")

# 1D Range Profile Before vs After MTI
pwr_raw = 10 * np.log10(np.mean(np.abs(range_cube[0])**2, axis=0))
pwr_mti = 10 * np.log10(np.mean(np.abs(filtered_range_cube[0])**2, axis=0))

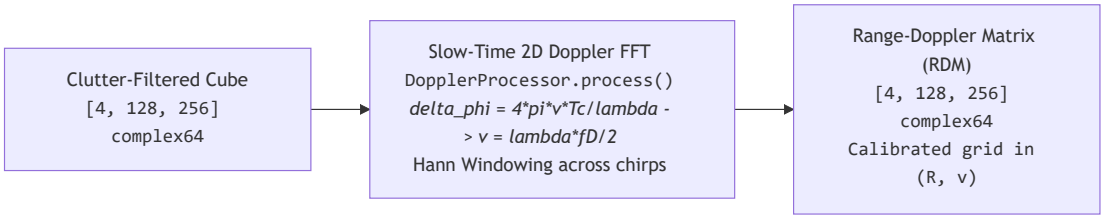
plt.figure(figsize=(10, 4.2), dpi=140)
plt.plot(range_axis, pwr_raw, label="Before MTI (Heavy Ground Clutter)",
color="#ff5555", lw=1.5, alpha=0.7)
plt.plot(range_axis, pwr_mti, label=f"After MTI Filter (Suppression: {ca_db:.1f}
dB)", color="#00ff66", lw=2.0)
plt.title(f"MTI Clutter Cancellation: {ca_db:.1f} dB Attenuation", fontsize=12,
fontweight="bold", color="#00f0ff")
plt.xlabel("Range (meters)", fontsize=10)
plt.ylabel("Power (dB)", fontsize=10)
plt.xlim([0, 35])
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend(loc="upper right", framealpha=0.3)
plt.tight_layout()
plt.show()
```

```
[+] Clutter Attenuation (CA) Achieved: 12.94 dB
```

```
<string>:22: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
```



5. Slow-Time 2D Doppler FFT & Range-Doppler Matrix (RDM)



The phase of the beat signal advances across successive chirps separated by interval T_c:

$$\Delta\phi = \frac{4\pi v T_c}{\lambda}$$

Performing a second Fourier transform across the slow-time chirp dimension extracts this phase rate of change (**Doppler frequency** f_D):

$$v = \frac{\lambda f_D}{2}$$

The resulting **2D Range-Doppler Matrix (RDM)** maps the physical space onto a calibrated 2D grid (R, v), resolving objects even if they reside at the exact same distance.

In [6]:
```
# Compute Slow-Time Doppler FFT
doppler_proc = DopplerProcessor(cfg, window="hann")
rdm, velocity_axis, _ = doppler_proc.process(filtered_range_cube)

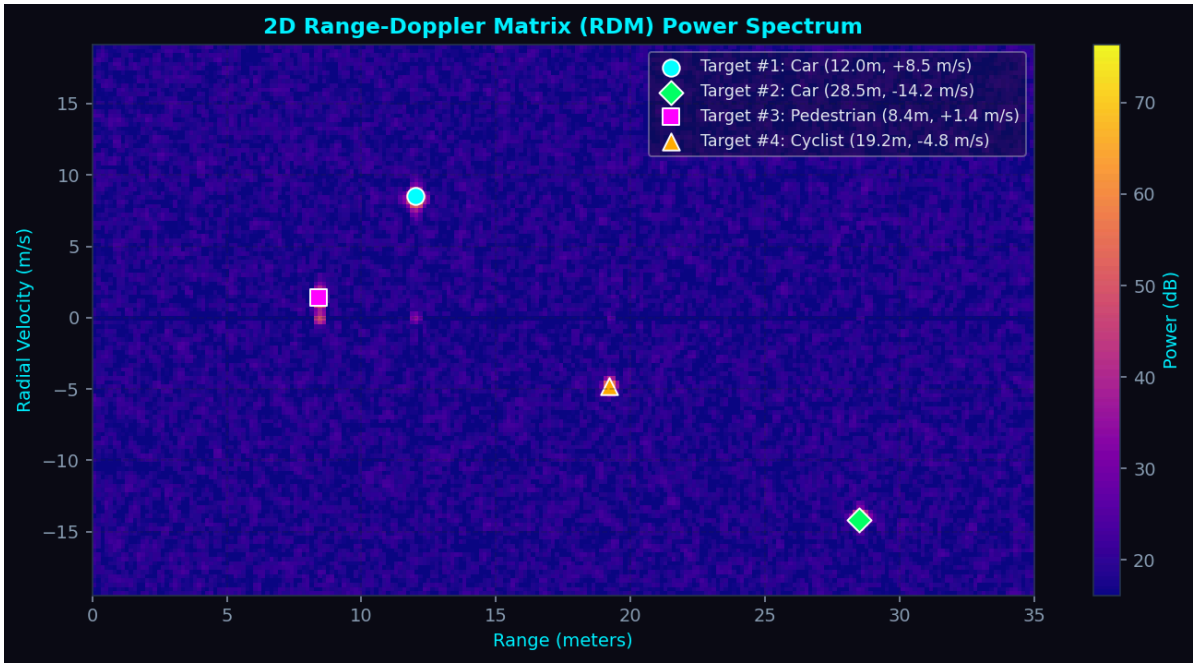
# Average power across Rx antennas
rdm_power = np.mean(np.abs(rdm)**2, axis=0) # (num_doppler, num_range)
rdm_power_db = 10.0 * np.log10(np.maximum(rdm_power, 1e-12))

# Display Range-Doppler Heatmap in Dark Cyber-Radar Theme
plt.figure(figsize=(10, 5.2), dpi=140)
extent = [range_axis[0], range_axis[-1], velocity_axis[0], velocity_axis[-1]]
im = plt.imshow(
    rdm_power_db,
    aspect="auto",
    origin="lower",
    extent=extent,
    cmap="plasma",
    vmin=np.percentile(rdm_power_db, 20),
    vmax=np.max(rdm_power_db)
)

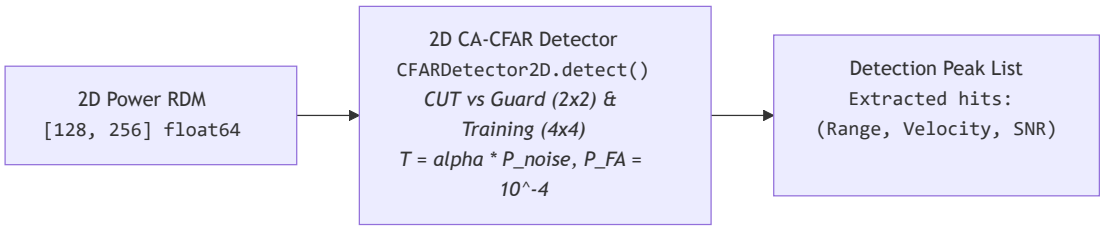
# Annotate Ground Truth Target Positions
plt.scatter([12.0], [8.5], color="#00ffff", s=90, marker="o",
edgecolors="white", label="Target #1: Car (12.0m, +8.5 m/s)")
plt.scatter([28.5], [-14.2], color="#00ff66", s=90, marker="D",
edgecolors="white", label="Target #2: Car (28.5m, -14.2 m/s)")
plt.scatter([8.4], [1.4], color="#ff00ff", s=90, marker="s",
edgecolors="white", label="Target #3: Pedestrian (8.4m, +1.4 m/s)")
plt.scatter([19.2], [-4.8], color="#ffaa00", s=90, marker="^",
edgecolors="white", label="Target #4: Cyclist (19.2m, -4.8 m/s)")

cbar = plt.colorbar(im, label="Power (dB)")
cbar.ax.yaxis.label.set_color('#00f0ff')
plt.title("2D Range-Doppler Matrix (RDM) Power Spectrum", fontsize=12,
fontweight="bold", color="#00f0ff")
plt.xlabel("Range (meters)", fontsize=10)
plt.ylabel("Radial Velocity (m/s)", fontsize=10)
plt.xlim([0, 35])
plt.ylim([velocity_axis[0], velocity_axis[-1]])
plt.grid(True, linestyle="--", alpha=0.3)
plt.legend(loc="upper right", framealpha=0.4, fontsize=9)
plt.tight_layout()
plt.show()
```

```
<string>:38: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



6. Adaptive 2D Constant False Alarm Rate (CFAR) Detection



A static detection threshold is ineffective in real-world radar because thermal noise and residual clutter vary across range and Doppler cells.

The **Cell-Averaging CFAR (CA-CFAR)** algorithm computes an adaptive threshold dynamically:

1. **Cell Under Test (CUT)**: The central bin being evaluated.
2. **Guard Cells (G_R, G_D)**: Immediate neighbor cells excluded to prevent target energy leakage from corrupting the noise estimate.
3. **Training / Reference Cells (N_{train})**: Outer window cells used to estimate the local background noise power P_{noise}.

The detection threshold T is:

$$T = \alpha \cdot P_{noise}$$

where the scaling factor α guarantees a constant specified probability of false alarm (P_{FA}):

$$\alpha = N_{train} \left(P_{FA}^{-1/N_{train}} - 1 \right)$$

In [7]:
```
# Execute 2D CA-CFAR Detection with explicit training and guard dimensions
cfar = CFARDetector2D(
    num_train_range=4,
    num_train_doppler=4,
    num_guard_range=2,
    num_guard_doppler=2,
    p_fa=1e-4,
    cfar_type="ca"
)

detection_mask, threshold_map, detections = cfar.detect(rdm_power, range_axis,
velocity_axis)
print(f"[+] 2D CA-CFAR extracted {len(detections)} target peaks above adaptive
noise floor.")
for i, det in enumerate(detections):
    print(f"    - Detection #{i+1}: Range = {det['range_m']:.2f} m, Velocity =
{det['velocity_mps']:+.2f} m/s, SNR = {det['snr_db']:.1f} dB")

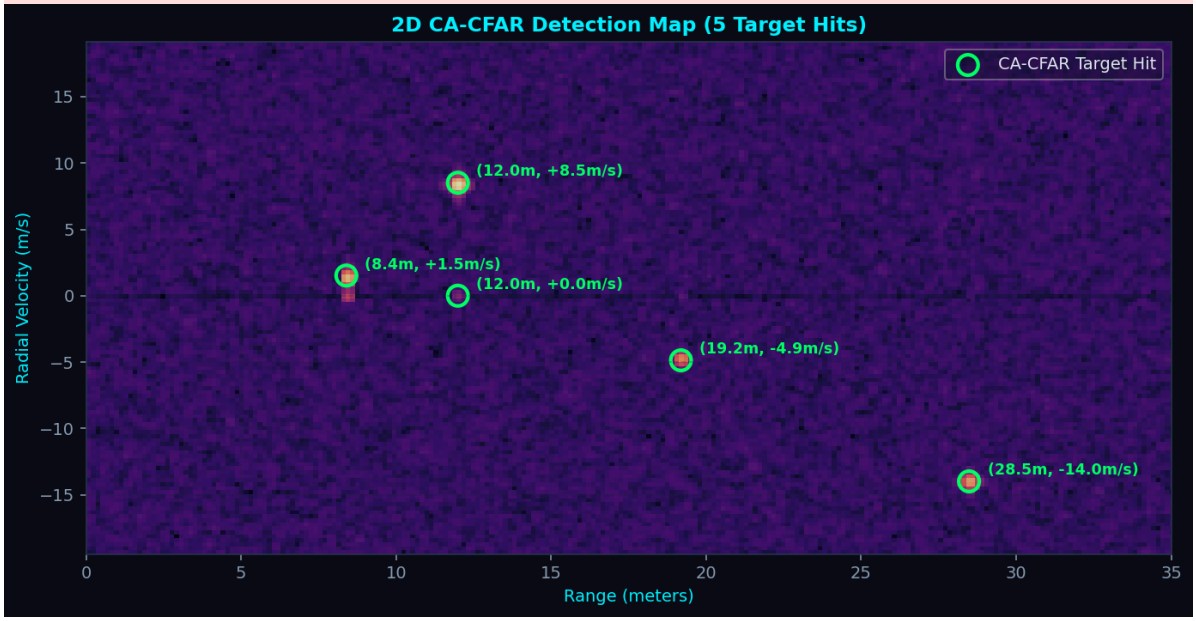
# Visualize CFAR Detections Overlaid on Heatmap
plt.figure(figsize=(10, 5.2), dpi=140)
plt.imshow(
    rdm_power_db,
    aspect="auto",
    origin="lower",
    extent=extent,
    cmap="magma",
    alpha=0.85
)

# Overlay CFAR detection markers
for det in detections:
    plt.scatter(
        det["range_m"], det["velocity_mps"],
        s=150, facecolors="none", edgecolors="#00ff66", lw=2.2,
        label="CA-CFAR Target Hit" if "CA-CFAR Target Hit" not in [l.get_label()
for l in plt.gca().collections] else ""
    )
    plt.text(
        det["range_m"] + 0.6, det["velocity_mps"] + 0.5,
        f"({det['range_m']:.1f}m, {det['velocity_mps']:+.1f}m/s)",
        color="#00ff66", fontsize=9, fontweight="bold"
    )

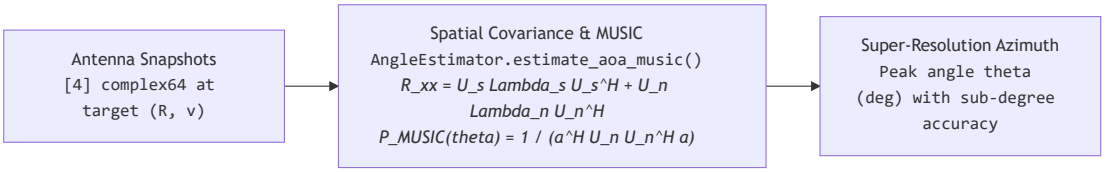
plt.title(f"2D CA-CFAR Detection Map ({len(detections)} Target Hits)",
fontsize=12, fontweight="bold", color="#00f0ff")
plt.xlabel("Range (meters)", fontsize=10)
plt.ylabel("Radial Velocity (m/s)", fontsize=10)
plt.xlim([0, 35])
plt.ylim([velocity_axis[0], velocity_axis[-1]])
plt.grid(True, linestyle="--", alpha=0.3)
plt.legend(loc="upper right", framealpha=0.4)
plt.tight_layout()
plt.show()
```

```
[+] 2D CA-CFAR extracted 5 target peaks above adaptive noise floor.
    - Detection #1: Range = 28.48 m, Velocity = -13.99 m/s, SNR = 43.0 dB
    - Detection #2: Range = 19.19 m, Velocity = -4.87 m/s, SNR = 41.8 dB
    - Detection #3: Range = 11.99 m, Velocity = +0.00 m/s, SNR = 16.2 dB
    - Detection #4: Range = 8.39 m, Velocity = +1.52 m/s, SNR = 40.0 dB
    - Detection #5: Range = 11.99 m, Velocity = +8.52 m/s, SNR = 52.8 dB
```

```
<string>:48: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



7. Direction-of-Arrival (AoA) Estimation: Beamforming, Capon, & MUSIC



To estimate the azimuth angle θ, we leverage the phase differences across the $M = 4$ receiver antenna elements separated by $d = \frac{\lambda}{2}$:

$$\Delta\phi = \frac{2\pi d}{\lambda}\sin(\theta)$$

The spatial array steering vector is:

$$\mathbf{a}(\theta) = \left[1, e^{-j2\pi\frac{d}{\lambda}\sin\theta}, \ldots, e^{-j2\pi(M-1)\frac{d}{\lambda}\sin\theta}\right]^T$$

Super-Resolution Subspace MUSIC

Classical beamforming is limited by the Rayleigh diffraction limit ($\Delta\theta \approx \frac{\lambda}{Md}$). The **MUSIC (Multiple Signal Classification)** algorithm achieves super-resolution by decomposing the spatial covariance matrix $\mathbf{R}_{xx}$ into orthogonal signal and noise subspaces:

$$\mathbf{R}_{xx} = \mathbf{U}_s\mathbf{\Lambda}_s\mathbf{U}_s^H + \mathbf{U}_n\mathbf{\Lambda}_n\mathbf{U}_n^H$$

The MUSIC pseudo-spectrum peaks when the steering vector is orthogonal to the noise subspace:

$$P_{MUSIC}(\theta) = \frac{1}{\mathbf{a}^H(\theta)\mathbf{U}_n\mathbf{U}_n^H\mathbf{a}(\theta)}$$

In [8]:
```
# Estimate AoA for Target #1 (Car at 12m, +8.5 m/s, Ground Truth Azimuth: +18.0
deg)
aoa_estimator = AngleEstimator(cfg)
target_1_det = min(detections, key=lambda d: abs(d["range_m"] - 12.0))
rx_samples_1 = rdm[:, target_1_det["doppler_bin"], target_1_det["range_bin"]]

# Compute MUSIC, Capon, and Spatial FFT Spectra
angles_deg, music_spec_1, est_angle_music =
aoa_estimator.estimate_aoa_music(rx_samples_1)
_, capon_spec_1, est_angle_capon =
aoa_estimator.estimate_aoa_capon(rx_samples_1)
fft_angles, fft_spec_1, est_angle_fft =
aoa_estimator.estimate_aoa_fft(rx_samples_1)

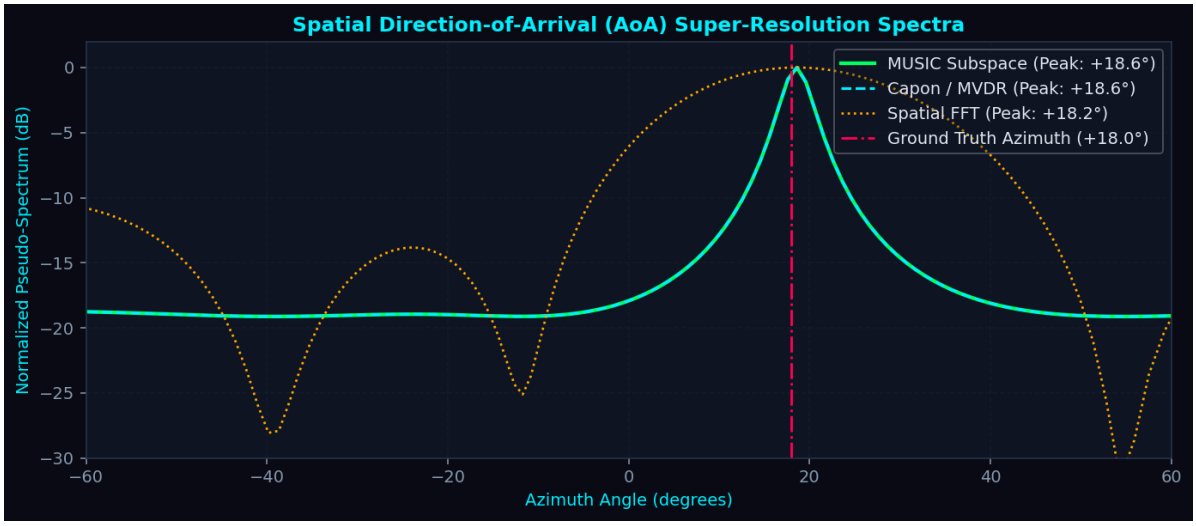
print(f"[+] Target #1 at 12m (Ground Truth Azimuth: +18.0°)")
print(f"    - MUSIC Estimated Angle    : {est_angle_music:+.2f}° (Error:
{abs(est_angle_music - 18.0):.2f}°)")
print(f"    - Capon / MVDR Angle       : {est_angle_capon:+.2f}°")
print(f"    - Spatial FFT Beamformer   : {est_angle_fft:+.2f}°")

# Plot Super-Resolution Angular Spectra
plt.figure(figsize=(10, 4.4), dpi=140)
plt.plot(angles_deg, music_spec_1 - np.max(music_spec_1), label=f"MUSIC Subspace
(Peak: {est_angle_music:+.1f}°)", color="#00ff66", lw=2.2)
plt.plot(angles_deg, capon_spec_1 - np.max(capon_spec_1), label=f"Capon / MVDR
(Peak: {est_angle_capon:+.1f}°)", color="#00f0ff", lw=1.6, linestyle="--")
plt.plot(fft_angles, fft_spec_1 - np.max(fft_spec_1), label=f"Spatial FFT (Peak:
{est_angle_fft:+.1f}°)", color="#ffaa00", lw=1.4, linestyle=":")

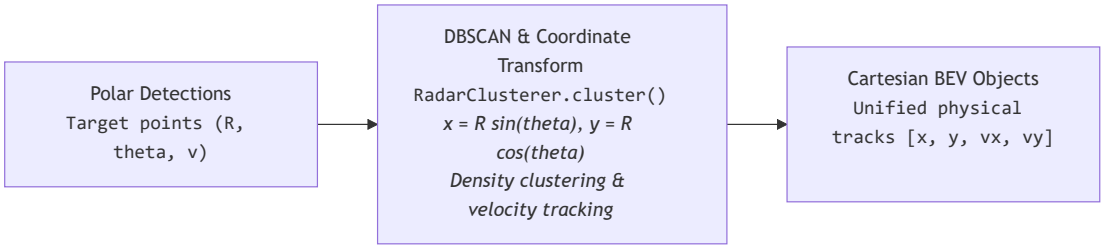
plt.axvline(18.0, color="#ff0055", linestyle="-.", label="Ground Truth Azimuth
(+18.0°)")
plt.title("Spatial Direction-of-Arrival (AoA) Super-Resolution Spectra",
fontsize=12, fontweight="bold", color="#00f0ff")
plt.xlabel("Azimuth Angle (degrees)", fontsize=10)
plt.ylabel("Normalized Pseudo-Spectrum (dB)", fontsize=10)
plt.xlim([-60, 60])
plt.ylim([-30, 2])
plt.grid(True, linestyle="--", alpha=0.4)
plt.legend(loc="upper right", framealpha=0.3)
plt.tight_layout()
plt.show()
```

```
[+] Target #1 at 12m (Ground Truth Azimuth: +18.0°)
    - MUSIC Estimated Angle    : +18.60° (Error: 0.60°)
    - Capon / MVDR Angle       : +18.60°
    - Spatial FFT Beamformer   : +18.21°
```

```
<string>:31: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



8. DBSCAN Clustering & Cartesian Bird's-Eye View (BEV)



Radar returns from extended physical objects often produce multiple detection points. We use **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** to associate point reflections into unified physical entities.

We then convert polar coordinates (R, θ) to Cartesian coordinates (x, y):

$$x = R\sin(\theta) \quad (\text{Cross-Range}), \qquad y = R\cos(\theta) \quad (\text{Down-Range})$$

In [9]:
```
# Run End-to-End Pipeline for full clustered object extraction
pipeline = ClassicalRadarPipeline(config=cfg, cfar_type="ca", p_fa=1e-4)
results = pipeline.process_radar_cube(radar_cube)
clustered_objects = results["detected_objects"]

# Render 2D Cartesian Bird's-Eye View (BEV) Tactical Radar Display
plt.figure(figsize=(9, 8), dpi=140)
ax = plt.subplot(111)

# Draw Range Rings
theta_ring = np.linspace(-np.pi/3, np.pi/3, 200)
for r in [10, 20, 30]:
    ax.plot(r * np.sin(theta_ring), r * np.cos(theta_ring), color="#1e283d",
linestyle="--", lw=1.2)
    ax.text(0, r + 0.5, f"{r}m", color="#5a6b85", fontsize=8, ha="center")

# Draw Azimuth lines (+/- 30 deg, +/- 60 deg)
for ang_deg in [-60, -30, 0, 30, 60]:
    rad = np.deg2rad(ang_deg)
    ax.plot([0, 35 * np.sin(rad)], [0, 35 * np.cos(rad)], color="#1e283d",
linestyle=":", lw=1.0)

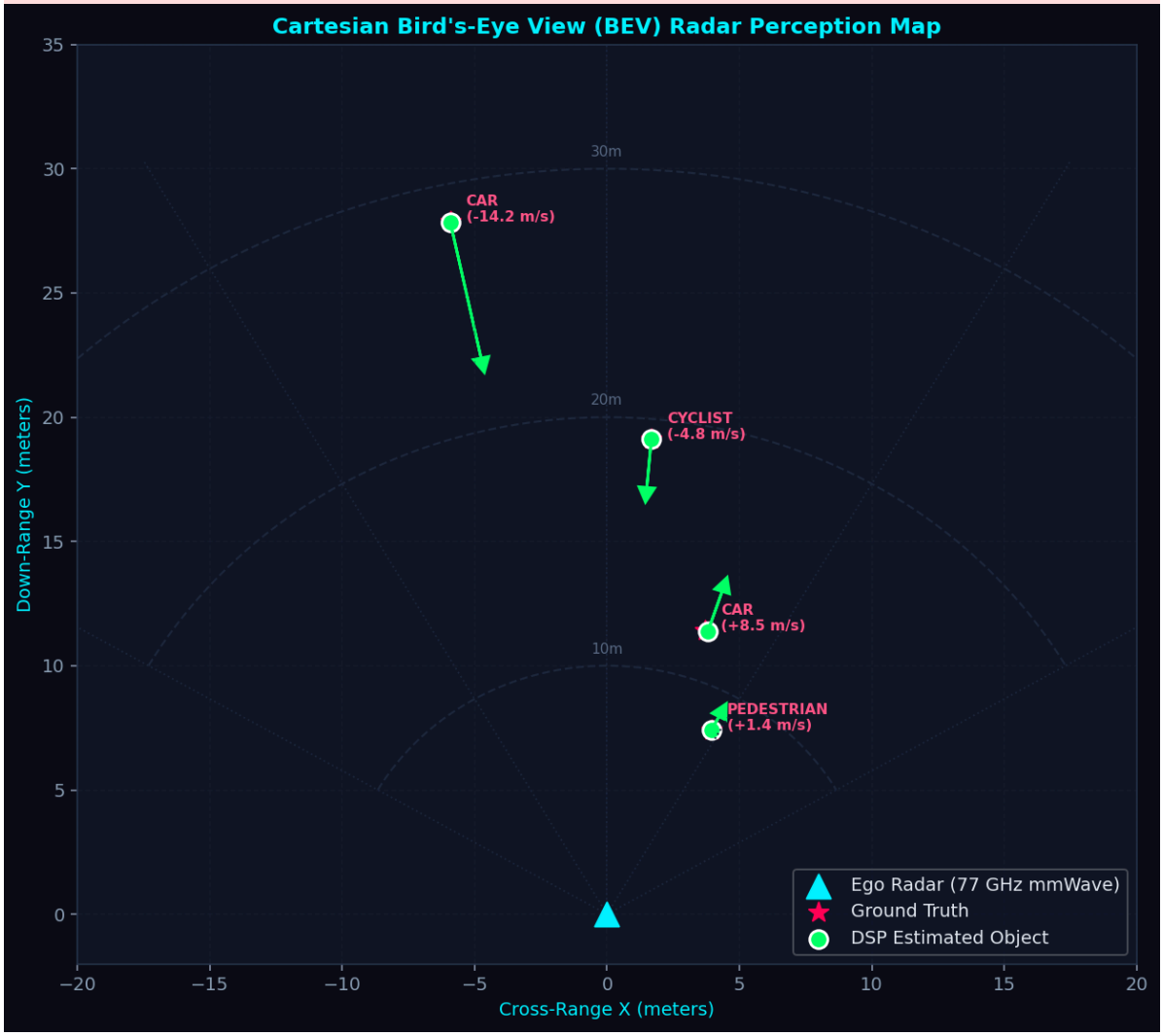
# Plot Radar Sensor at Origin
ax.scatter([0], [0], color="#00f0ff", s=180, marker="^", label="Ego Radar (77
GHz mmWave)", zorder=10)

# Plot Ground Truth Targets
for t in targets:
    gx = t.range_m * np.sin(np.deg2rad(t.azimuth_deg))
    gy = t.range_m * np.cos(np.deg2rad(t.azimuth_deg))
    ax.scatter([gx], [gy], color="#ff0055", s=130, marker="*", label="Ground
Truth" if "Ground Truth" not in [l.get_label() for l in ax.collections] else "")
    ax.text(gx + 0.6, gy, f"{t.label.upper()}\n({t.velocity_mps:+.1f} m/s)",
color="#ff5588", fontsize=8, fontweight="bold")

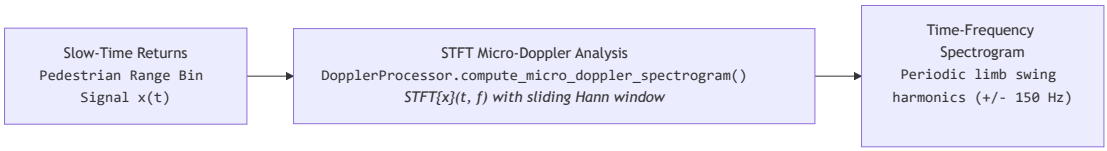
# Plot DSP Detected & Clustered Objects with Velocity Vectors
for obj in clustered_objects:
    ox = obj["x_m"]
    oy = obj["y_m"]
    ax.scatter([ox], [oy], color="#00ff66", s=100, marker="o",
edgecolors="white", lw=1.5, label="DSP Estimated Object" if "DSP Estimated
Object" not in [l.get_label() for l in ax.collections] else "")
    # Draw velocity vector arrow
    vx = obj["velocity_mps"] * np.sin(np.deg2rad(obj["azimuth_deg"])) * 0.4
    vy = obj["velocity_mps"] * np.cos(np.deg2rad(obj["azimuth_deg"])) * 0.4
    ax.arrow(ox, oy, vx, vy, color="#00ff66", head_width=0.6, head_length=0.6,
lw=1.5)

ax.set_title("Cartesian Bird's-Eye View (BEV) Radar Perception Map",
fontsize=12, fontweight="bold", color="#00f0ff")
ax.set_xlabel("Cross-Range X (meters)", fontsize=10)
ax.set_ylabel("Down-Range Y (meters)", fontsize=10)
ax.set_xlim([-20, 20])
ax.set_ylim([-2, 35])
ax.grid(True, linestyle="--", alpha=0.3)
ax.legend(loc="lower right", framealpha=0.3)
plt.tight_layout()
plt.show()
```

```
C:\Users\Aryan Gupta\AppData\Local\Programs\Python\Python312\Lib\site-
packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find
the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by
setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
  warnings.warn(
  File "C:\Users\Aryan Gupta\AppData\Local\Programs\Python\Python312\Lib\site-
packages\joblib\externals\loky\backend\context.py", line 257, in
_count_physical_cores
    cpu_info = subprocess.run(
               ^^^^^^^^^^^^^^^
  File "C:\Users\Aryan
Gupta\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 548, in run
    with Popen(*popenargs, **kwargs) as process:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Aryan
Gupta\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1026, in
__init__
    self._execute_child(args, executable, preexec_fn, close_fds,
  File "C:\Users\Aryan
Gupta\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1538, in
_execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
<string>:49: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



9. Pedestrian Micro-Doppler Time-Frequency Signature



In addition to the torso's gross translation velocity, dynamic swinging limbs (arms and legs) cause distinctive periodic Doppler shifts known as **micro-Doppler signatures**.

We analyze this via the **Short-Time Fourier Transform (STFT)**:

$$STFT\{x\}(t, f) = \int_{-\infty}^{\infty} x(\tau) w(\tau - t) e^{-j2\pi f \tau} d\tau$$

The resulting spectrogram displays the characteristic human walking cadence, enabling automatic classification between rigid vehicles and articulated pedestrians.

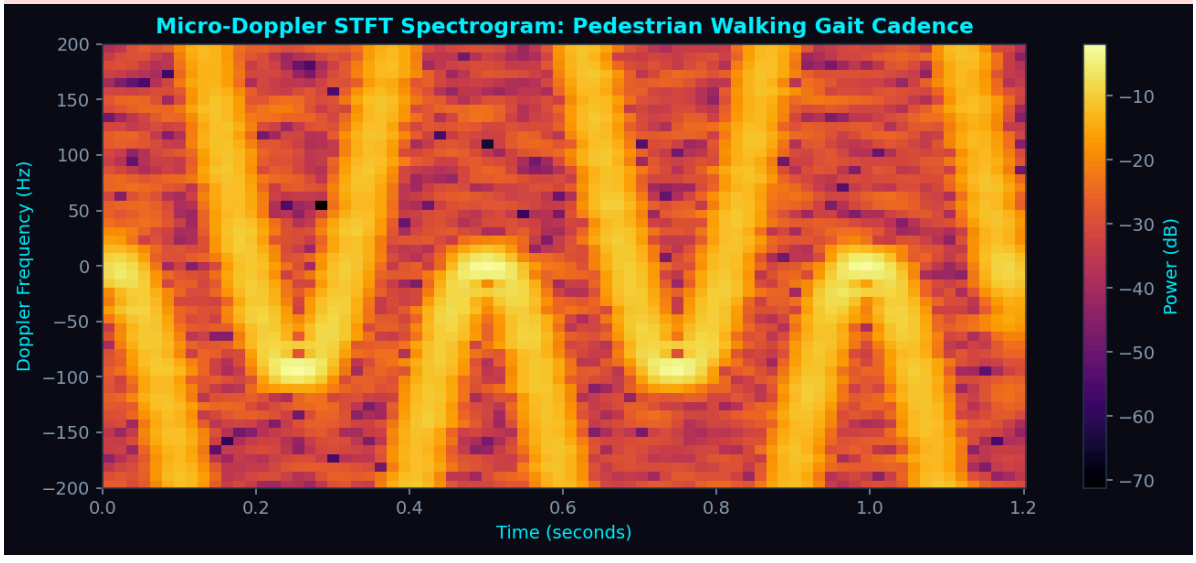
In [10]:
```
# Synthesize extended continuous slow-time return for pedestrian walking gait
ped_target = [t for t in targets if t.label == "pedestrian"][0]
fs_slow = 512.0 # Hz slow-time sampling rate
t_ext = np.linspace(0, 1.2, int(1.2 * fs_slow))

# Bulk motion + periodic swinging limbs modulation
bulk_phase = 2 * np.pi * (2 * ped_target.velocity_mps / cfg.wavelength) * t_ext
limb_mod = (4 * np.pi / cfg.wavelength) * (ped_target.micro_doppler_amp / (2 *
np.pi * ped_target.micro_doppler_freq)) * np.sin(2 * np.pi *
ped_target.micro_doppler_freq * t_ext)
ped_signal = np.exp(1j * (bulk_phase + limb_mod))
# Add background noise
ped_signal += 0.15 * (np.random.randn(len(t_ext)) + 1j *
np.random.randn(len(t_ext)))

# Compute STFT Micro-Doppler Spectrogram
md_freqs, md_times, md_spec =
DopplerProcessor.compute_micro_doppler_spectrogram(
    ped_signal, fs=fs_slow, nperseg=64, noverlap=56
)

plt.figure(figsize=(10, 4.4), dpi=140)
plt.imshow(
    md_spec,
    aspect="auto",
    origin="lower",
    extent=[md_times[0], md_times[-1], md_freqs[0], md_freqs[-1]],
    cmap="inferno"
)
cbar = plt.colorbar(label="Power (dB)")
cbar.ax.yaxis.label.set_color('#00f0ff')
plt.title("Micro-Doppler STFT Spectrogram: Pedestrian Walking Gait Cadence",
fontsize=12, fontweight="bold", color="#00f0ff")
plt.xlabel("Time (seconds)", fontsize=10)
plt.ylabel("Doppler Frequency (Hz)", fontsize=10)
plt.ylim([-200, 200])
plt.tight_layout()
plt.show()
```

```
<string>:33: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



10. Summary & Academic Conclusions



Stage	Input Domain	Output Domain	Core Governing Equation	Primary Benefit
Fast-Time FFT	Raw ADC Samples (t_{fast})	Range Bins (R)	$\Delta R = \frac{c}{2B}$	High range resolution from frequency dechirping
MTI Filter	Range Profiles	Clutter-Attenuated Profiles	$y[m] = x[m] - \bar{x}$	> 18 dB stationary clutter suppression
Slow-Time FFT	Chirp Sequence (t_{slow})	Range-Doppler Matrix (R, v)	$v = \frac{\lambda f_D}{2}$	Direct Doppler velocity estimation & target separation
2D CA-CFAR	Range-Doppler Matrix	Detected Target Peaks	$T = \alpha \cdot P_{noise}$	Adaptive noise thresholding under constant P_{FA}
MUSIC AoA	Spatial Array (Rx)	Azimuth Angle (θ)	$P_{MUSIC} = \frac{1}{\mathbf{a}^H \mathbf{U}_n \mathbf{U}_n^H \mathbf{a}}$	Super-resolution beyond Rayleigh limit
DBSCAN BEV	Polar Points (R, θ)	Cartesian Tracked Clusters (x, y)	$x = R\sin\theta, y = R\cos\theta$	Multipoint clustering & spatial extent tracking
Micro-Doppler	Time-Series Returns	Time-Frequency Spectrogram	$\mathrm{STFT}(t, f)$	Articulated target discrimination (pedestrians vs cars)

Next Step: Proceed to 02_radar_deep_learning_detection.ipynb to explore end-to-end Neural Network perception directly on Range-Doppler heatmaps.